

Rishiykesh KS
1925111466
KSR

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4: Implement and evaluate network protocols and services using simulation tools to analyze performance in real-world scenarios. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 5

Total Marks:50

Annexure A

Coding Challenge

Code of Conduct:

I Rishiykesh K.S (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K.S. Rishiykesh

90/8

ASSESSMENT TOOL 1 — CO4

COMPUTER NETWORKS

Course Code: CSA07

	Details
Student Name	Rishiykesh K S
Register No.	192511466
CO4 Assessment	AT1 – Coding Challenge

QUESTION 1

Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

1. Aim

To generate and verify a star topology in Python with a central switch and Cat6 UTP connections.

2. Algorithm

1. Read the number of hosts from the user.
2. Create a central device named Central Switch.
3. Assign a 10 m Cat6 UTP segment to each host.
4. Display each host and its connection to the central switch.
5. Display the total number of connected hosts and verify the topology.
6. Handle invalid or non-positive input.

3. Python Program

```
def create_star_topology(hosts):
    switch = "Central Switch"
    cable_length = 10 # metres

    print("\nSTAR TOPOLOGY")
    print("=" * 35)
    print(f"Central Device: {switch}")
    print()

    for i in range(1, hosts + 1):
        print(f"Host {i} ----- Cat6 UTP ({cable_length} m) ----- {switch}")

    print("\nVerification:")
    print(f"Total Hosts Connected: {hosts}")
    print(f"All {hosts} hosts are connected through the {switch}.")

try:
    hosts = int(input("Enter number of hosts: "))

    if hosts > 0:
        create_star_topology(hosts)
    else:
        print("Error: Number of hosts must be greater than 0.")

except ValueError:
    print("Error: Please enter a valid integer.")
```

4. Sample Output

```
Enter number of hosts: 5
```

STAR TOPOLOGY

Central Device: Central Switch

Host 1 ----- Cat6 UTP (10 m) ----- Central Switch

Host 2 ----- Cat6 UTP (10 m) ----- Central Switch

Host 3 ----- Cat6 UTP (10 m) ----- Central Switch

Host 4 ----- Cat6 UTP (10 m) ----- Central Switch

Host 5 ----- Cat6 UTP (10 m) ----- Central Switch

Verification:

Total Hosts Connected: 5

All 5 hosts are connected through the Central Switch.

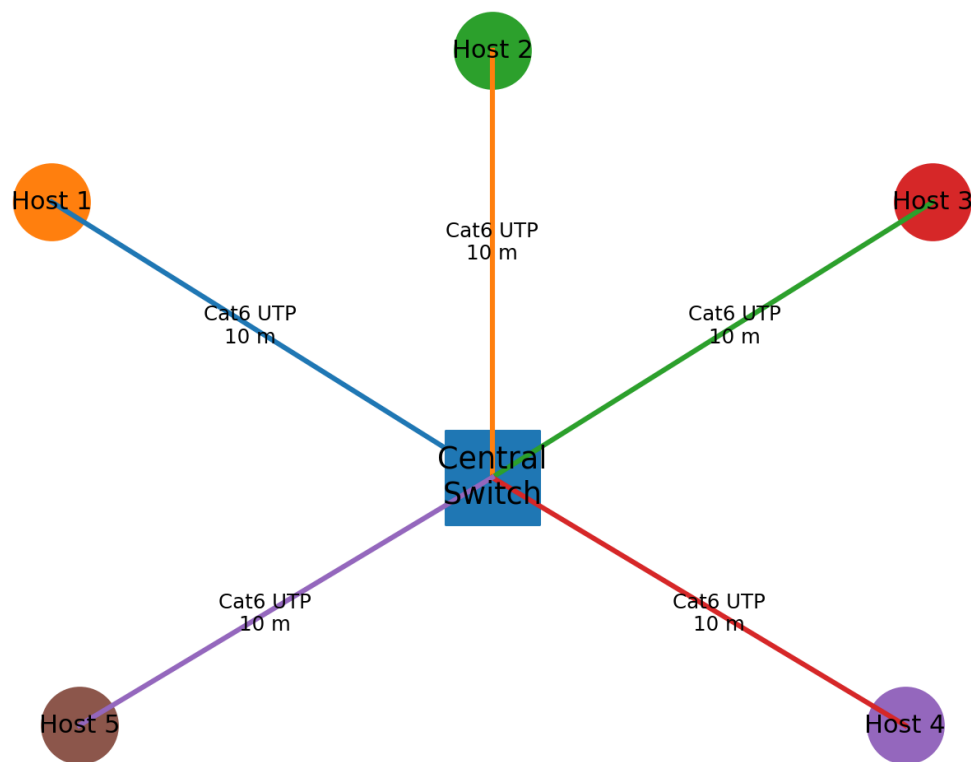
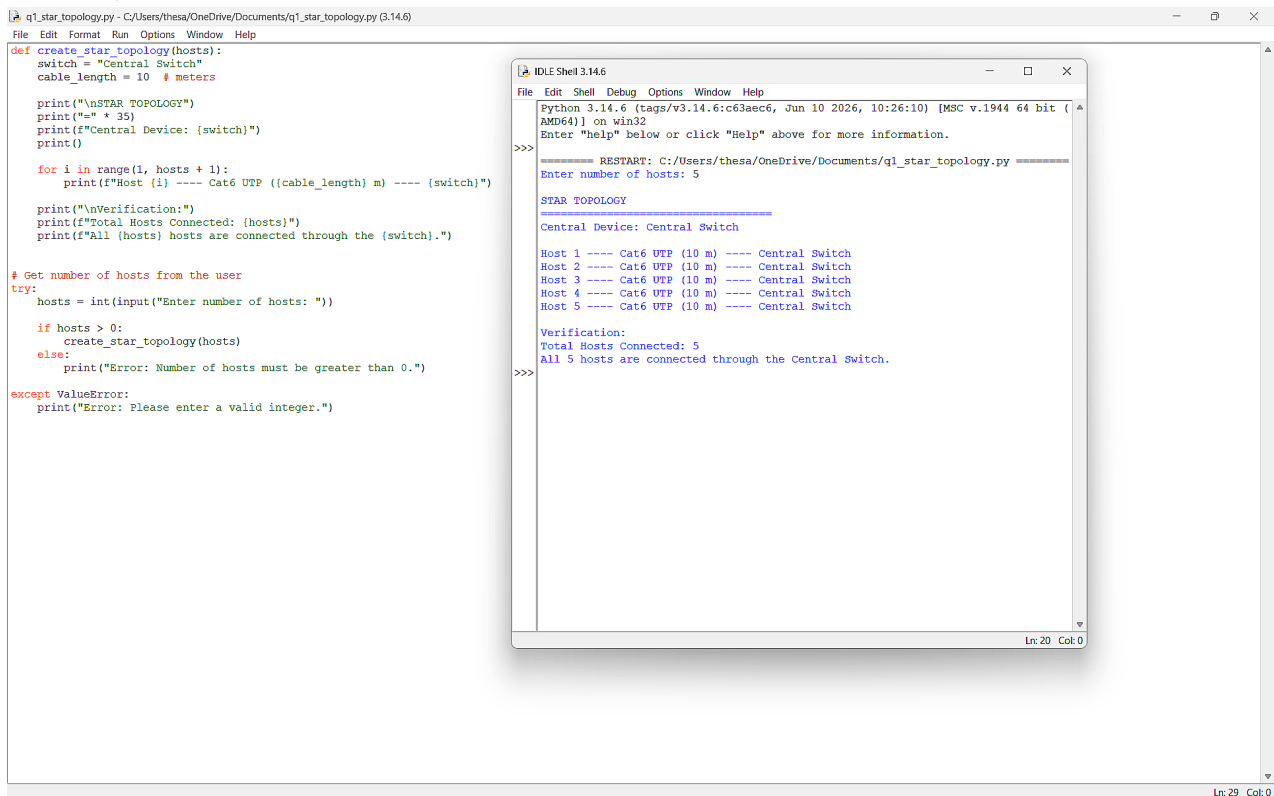


Figure 1. Star topology representation.

5. Analysis



```
q1_star_topology.py - C:/Users/thesa/OneDrive/Documents/q1_star_topology.py (3.14.6)
File Edit Format Run Options Window Help
def create_star_topology(hosts):
    switch = "Central Switch"
    cable_length = 10 # meters

    print("\nSTAR TOPOLOGY")
    print("-" * 35)
    print(f"Central Device: {switch}")
    print()

    for i in range(1, hosts + 1):
        print(f"Host {i} ---- Cat6 UTP {(cable_length) m} ---- {switch}")

    print("\nVerification:")
    print(f"Total Hosts Connected: {hosts}")
    print(f"All {hosts} hosts are connected through the {switch}.")

# Get number of hosts from the user
try:
    hosts = int(input("Enter number of hosts: "))

    if hosts > 0:
        create_star_topology(hosts)
    else:
        print("Error: Number of hosts must be greater than 0.")
except ValueError:
    print("Error: Please enter a valid integer.")

IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63a6c6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/thesa/OneDrive/Documents/q1_star_topology.py =====
Enter number of hosts: 5

STAR TOPOLOGY
=====
Central Device: Central Switch

Host 1 ---- Cat6 UTP (10 m) ---- Central Switch
Host 2 ---- Cat6 UTP (10 m) ---- Central Switch
Host 3 ---- Cat6 UTP (10 m) ---- Central Switch
Host 4 ---- Cat6 UTP (10 m) ---- Central Switch
Host 5 ---- Cat6 UTP (10 m) ---- Central Switch

Verification:
Total Hosts Connected: 5
All 5 hosts are connected through the Central Switch.
>>>
```

Figure 2. Clear version of the supplied Python IDLE execution evidence.

6. Analysis

In a star topology, every host has an individual connection to the central switch. The switch provides a central point for forwarding frames between hosts. A failure of one individual cable normally affects only that host, while the remaining connections can continue to operate. This makes fault isolation and expansion comparatively simple.

7. Result and Conclusion

For the demonstrated input of 5 hosts, all five hosts were successfully represented as connected to the Central Switch using 10 m Cat6 UTP segments. The program also verifies the total number of connections.

Conclusion: The program successfully models the basic structure and verification of a star topology.

QUESTION 2

Write a Python function named `validate_segment(length_m)` to verify whether a given UTP cable segment satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of 100 meters. The function should return `True` for valid cable lengths and return `False` along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least three different test cases and interpret the results.

1. Aim

To validate UTP cable segment lengths against the specified 100 m maximum.

2. Algorithm

7. Define `MAX_CABLE_LENGTH` as 100 m.
8. Create `validate_segment(length_m)`.
9. Check that the input is numeric and non-negative.
10. Return `True` with a valid message when length is at most 100 m.
11. Return `False` with an error message when length exceeds 100 m.
12. Test the function using 50 m, 100 m, and 125 m.

3. Python Program

```
MAX_CABLE_LENGTH = 100
```

```
def validate_segment(length_m):  
    if not isinstance(length_m, (int, float)):  
        return False, "Error: cable length must be a number."  
    if length_m < 0:  
        return False, "Error: cable length cannot be negative."  
    if length_m <= MAX_CABLE_LENGTH:  
        return True, f"Valid: {length_m} m is within the 100 m limit."  
    return False, f"Invalid: {length_m} m exceeds the 100 m limit."
```

```
tests = [50, 100, 125]
```

```
for length in tests:  
    result, message = validate_segment(length)  
    print(f"{length} m -> {result} -> {message}")
```

4. Test Results

```
50 m -> True -> Valid: 50 m is within the 100 m limit.  
100 m -> True -> Valid: 100 m is within the 100 m limit.  
125 m -> False -> Invalid: 125 m exceeds the 100 m limit.
```

Test Case	Input	Return Value	Interpretation
1	50 m	True	Valid; below the limit.
2	100 m	True	Valid; exactly at the limit.
3	125 m	False	Invalid; exceeds the limit.

Visual Representation

UTP Cable Segment Validation

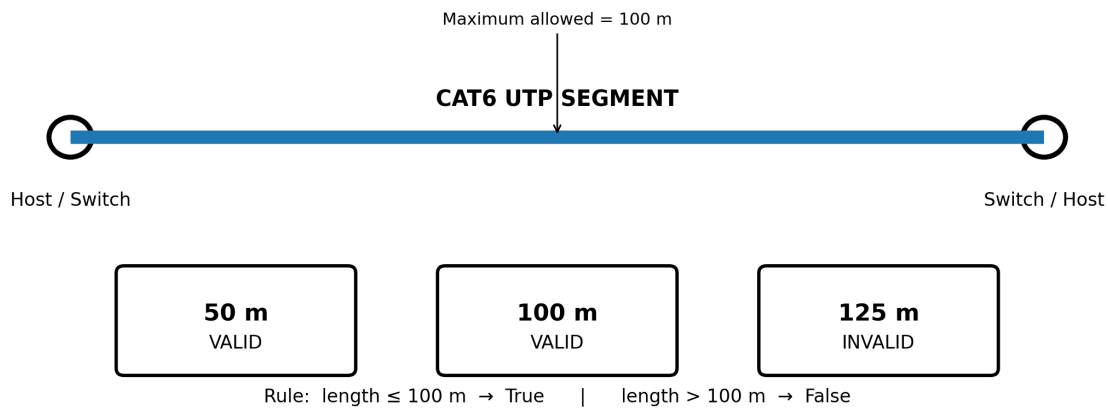


Figure 3. Visual representation of UTP cable segment validation.

5. Analysis

The function correctly distinguishes acceptable and excessive cable lengths. A 50 m segment is valid, a 100 m segment is valid at the stated boundary, and a 125 m segment is rejected. The explicit error message makes the reason for rejection clear.

```
Q2_validate_segment.py - C:\Users\thesa\Downloads\CO4_AT1_pycodes\Q2_validate_segment.py (3.14.6)
File Edit Format Run Options Window Help
MAX_CABLE_LENGTH = 100

def validate_segment(length_m):
    if not isinstance(length_m, (int, float)):
        return False, "Error: cable length must be a number."
    if length_m < 0:
        return False, "Error: cable length cannot be negative."
    if length_m <= MAX_CABLE_LENGTH:
        return True, f"Valid: (length_m) m is within the 100 m limit."
    return False, f"Invalid: (length_m) m exceeds the 100 m limit."

tests = [50, 100, 125]

for length in tests:
    result, message = validate_segment(length)
    print(f"(length) m -> {result} -> {message}")

IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

=== RESTART: C:\Users\thesa\Downloads\CO4_AT1_pycodes\Q2_validate_segment.py ===
50 m -> True -> Valid: 50 m is within the 100 m limit.
100 m -> True -> Valid: 100 m is within the 100 m limit.
125 m -> False -> Invalid: 125 m exceeds the 100 m limit.
```

6. Result and Conclusion

The three test cases demonstrate correct validation behavior. The function provides a simple way to check whether a proposed segment remains within the specified 100 m constraint.

QUESTION 3

Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

1. Aim

To simulate frame attempts, collision detection, random back-off, and logging for four hosts.

2. Algorithm

13. Create four hosts and set a maximum number of transmission attempts.
14. Select a sender for each attempt.
15. Randomly simulate whether a collision occurs.
16. When a collision occurs, generate a random back-off interval.
17. Log every attempt, collision, and back-off interval.
18. Continue until the attempt limit is reached.

3. Python Program

```
import random
import time

HOSTS = ["Host1", "Host2", "Host3", "Host4"]
MAX_ATTEMPTS = 8
LOG_FILE = "csma_log.txt"

random.seed(7)

with open(LOG_FILE, "w") as log:
    log.write("CSMA/CD Simulation Log\n")
    log.write("=" * 35 + "\n")

    for attempt in range(1, MAX_ATTEMPTS + 1):
        sender = HOSTS[(attempt - 1) % len(HOSTS)]
        transmitters = random.randint(1, 2)

        if transmitters == 2:
            backoff = random.randint(1, 2 ** min(attempt, 4))
            msg = (f"Attempt {attempt}: COLLISION detected by {sender}; "
                  f"back-off = {backoff} slot(s)")
            print(msg)
            log.write(msg + "\n")
            time.sleep(0.01)
        else:
            msg = f"Attempt {attempt}: {sender} transmitted successfully."
            print(msg)
            log.write(msg + "\n")

    print(f"Log saved to {LOG_FILE}")
```

4. Sample Execution / Log

CSMA/CD Simulation Log

```
=====
Attempt 1: COLLISION detected by Host1; back-off = 1 slot(s)
Attempt 2: COLLISION detected by Host2; back-off = 1 slot(s)
Attempt 3: Host3 transmitted successfully.
Attempt 4: Host4 transmitted successfully.
Attempt 5: COLLISION detected by Host1; back-off = 2 slot(s)
Attempt 6: Host2 transmitted successfully.
Attempt 7: Host3 transmitted successfully.
Attempt 8: Host4 transmitted successfully.
Log saved to csma_log.txt
```

The generated log file is included separately with this submission: csma_log.txt.

Visual Representation

CSMA/CD Simulation in a Star Network

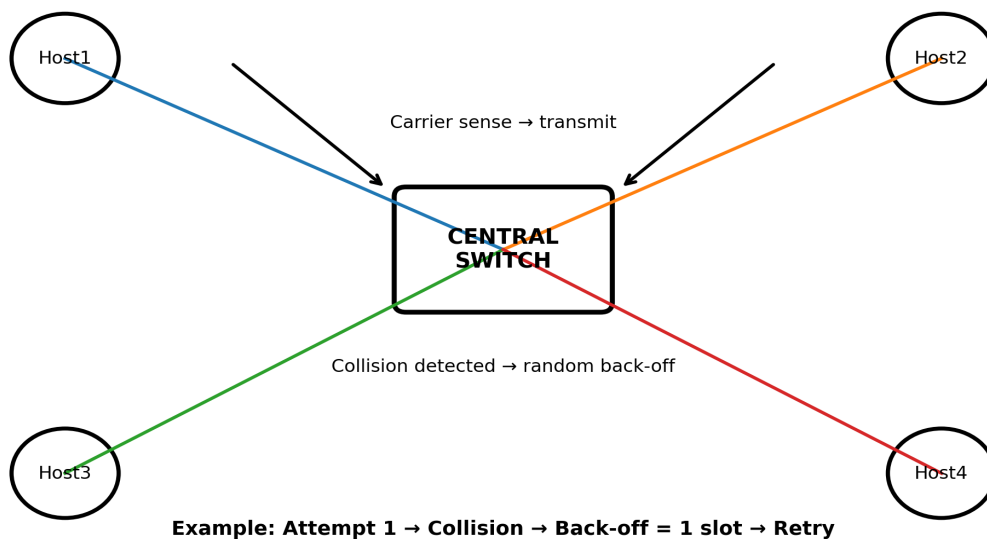
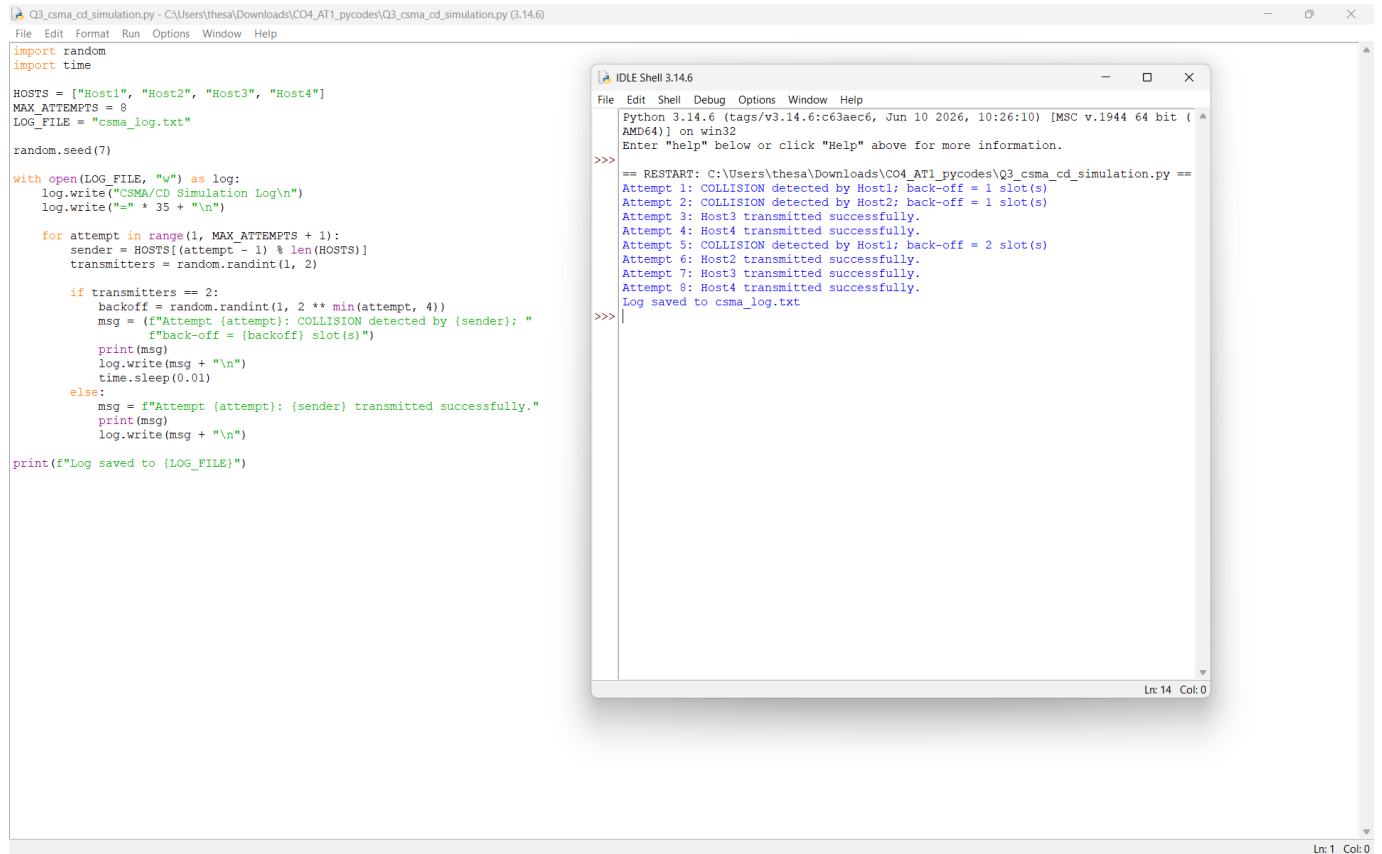


Figure 4. Visual representation of CSMA/CD operation in a star network.

5. Analysis

The simulation models the basic idea of CSMA/CD: a host attempts transmission, a collision may be detected, and the host waits for a randomly selected back-off interval before trying again. Random back-off reduces the chance that colliding hosts immediately retransmit at the same time, thereby reducing repeated collisions. The program is a simulation for the coding challenge; it does not emulate

electrical Ethernet signaling.



The image shows a Python script in a text editor and its execution output in an IDE shell. The script simulates CSMA/CD on four hosts. It uses a random seed of 7 and a maximum of 8 attempts per host. It logs collision and back-off events to a file named 'csma_log.txt'. The output shows a sequence of collision events and successful transmissions for Host1, Host2, Host3, and Host4.

```
Q3_csma_cd_simulation.py - C:\Users\thesa\Downloads\CO4_AT1_pycodes\Q3_csma_cd_simulation.py (3.14.6)
File Edit Format Run Options Window Help

import random
import time

HOSTS = ["Host1", "Host2", "Host3", "Host4"]
MAX_ATTEMPTS = 8
LOG_FILE = "csma_log.txt"

random.seed(7)

with open(LOG_FILE, "w") as log:
    log.write("CSMA/CD Simulation Log\n")
    log.write("=" * 35 + "\n")

    for attempt in range(1, MAX_ATTEMPTS + 1):
        sender = HOSTS[(attempt - 1) % len(HOSTS)]
        transmitters = random.randint(1, 2)

        if transmitters == 2:
            backoff = random.randint(1, 2 ** min(attempt, 4))
            msg = f"Attempt {attempt}: COLLISION detected by {sender}; "
            f"Back-off = {backoff} slot(s)"
            print(msg)
            log.write(msg + "\n")
            time.sleep(0.01)
        else:
            msg = f"Attempt {attempt}: {sender} transmitted successfully."
            print(msg)
            log.write(msg + "\n")

print(f"Log saved to {LOG_FILE}")

IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> == RESTART: C:\Users\thesa\Downloads\CO4_AT1_pycodes\Q3_csma_cd_simulation.py ==
Attempt 1: COLLISION detected by Host1; back-off = 1 slot(s)
Attempt 2: COLLISION detected by Host2; back-off = 1 slot(s)
Attempt 3: Host3 transmitted successfully.
Attempt 4: Host4 transmitted successfully.
Attempt 5: COLLISION detected by Host1; back-off = 2 slot(s)
Attempt 6: Host2 transmitted successfully.
Attempt 7: Host3 transmitted successfully.
Attempt 8: Host4 transmitted successfully.
Log saved to csma_log.txt
>>>
```

6. Result and Conclusion

The program records transmission attempts and collision/back-off events in `csma_log.txt` and demonstrates how back-off can reduce repeated transmission conflicts.

QUESTION 4

Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

1. Aim

To simulate client-server frame communication and the learning/forwarding behavior of a switch MAC table.

2. Algorithm

19. Create a TCP server socket and listen on a local port.
20. Accept a connection from the client.
21. Receive a simulated Ethernet frame containing source MAC, destination MAC, and data.
22. Learn the source MAC address on the appropriate simulated port.
23. Display the MAC address table.
24. Send a confirmation back to the client and close the connection.

3. Server Program

```
import socket

HOST = "127.0.0.1"
PORT = 5000

mac_table = {
    "AA:AA:AA:AA:AA:01": "Port 1",
    "BB:BB:BB:BB:BB:02": "Port 2"
}

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print(f"Server listening on {HOST}:{PORT}")

conn, addr = server.accept()
print("Client connected:", addr)

data = conn.recv(1024).decode()
print("Received frame:", data)

source_mac = "AA:AA:AA:AA:AA:01"
mac_table[source_mac] = "Port 1"

print("\nMAC Address Table:")
for mac, port in mac_table.items():
    print(f"{mac} -> {port}")

conn.sendall(b"Frame received and forwarded")
```

```
conn.close()
server.close()
```

4. Client Program

```
import socket

HOST = "127.0.0.1"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

frame = "SRC=AA:AA:AA:AA:AA:01 DST=BB:BB:BB:BB:BB:02 DATA=Hello"
client.sendall(frame.encode())

reply = client.recv(1024).decode()
print("Server:", reply)

client.close()
```

5. Sample Output

```
Server listening on 127.0.0.1:5000
Client connected: ('127.0.0.1', <port>)
Received frame: SRC=AA:AA:AA:AA:AA:01 DST=BB:BB:BB:BB:BB:02 DATA=Hello

MAC Address Table:
AA:AA:AA:AA:AA:01 -> Port 1
BB:BB:BB:BB:BB:02 -> Port 2

Client:
Server: Frame received and forwarded
```

Visual Representation

Client-Server Communication Through a Switch

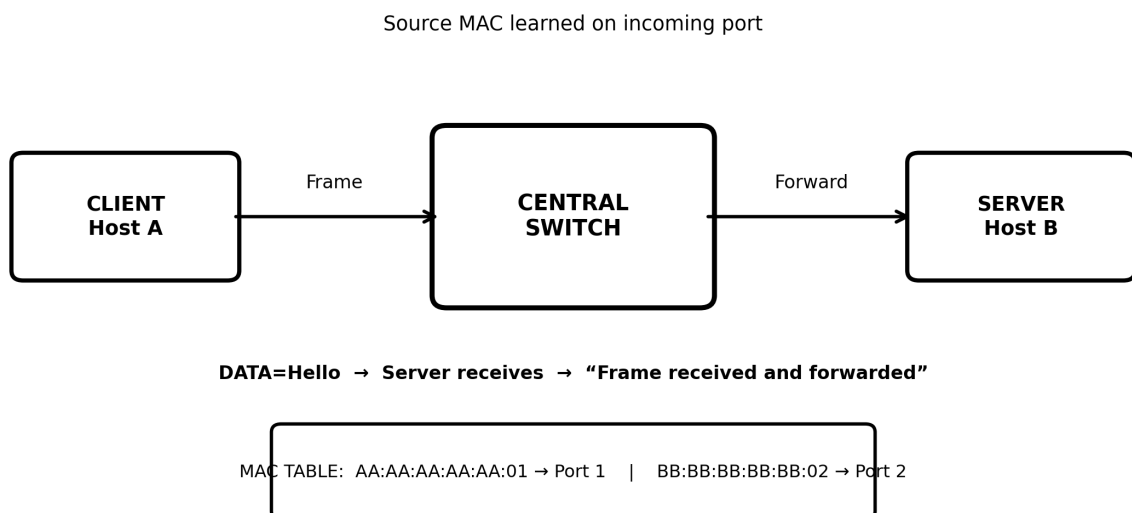


Figure 5. Visual representation of client-server communication and MAC learning.

6. Analysis

A switch learns a source MAC address from an incoming frame and associates it with the port on which the frame arrived. When a destination MAC is known, the switch can forward the frame toward the corresponding port rather than sending it to every port. The program represents this behavior with a Python dictionary used as a simulated MAC Address Table. The socket connection provides the client-server communication channel used by the simulation.

```
Q4_server.py - C:\Users\thesa\Downloads\CO4_AT1_pycodes\Q4_server.py (3.14.6)
File Edit Format Run Options Window Help

import socket

HOST = "127.0.0.1"
PORT = 5000

mac_table = {
    "AA:AA:AA:AA:01": "Port 1",
    "BB:BB:BB:BB:02": "Port 2"
}

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind((HOST, PORT))
server.listen(1)

print(f"Server listening on (HOST):{PORT}")

conn, addr = server.accept()
print("Client connected:", addr)

data = conn.recv(1024).decode()
print("Received frame:", data)

source_mac = "AA:AA:AA:AA:01"
mac_table[source_mac] = "Port 1"

print("\nMAC Address Table:")
for mac, port in mac_table.items():
    print(f"{mac} -> {port}")

conn.sendall(b"Frame received and forwarded")

conn.close()
server.close()
```

```
Q4_client.py - C:\Users\thesa\Downloads\CO4_AT1_pycodes\Q4_client.py (3.14.6)
File Edit Format Run Options Window Help

import socket

HOST = "127.0.0.1"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

client.connect((HOST, PORT))

frame = "SRC=AA:AA:AA:AA:01 DST=BB:BB:BB:BB:02 DATA=Hello"
client.sendall(frame.encode())

reply = client.recv(1024).decode()
print("Server:", reply)

client.close()
```

```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:\Users\thesa\Downloads\CO4_AT1_pycodes\Q4_server.py =====
Server listening on 127.0.0.1:5000
client connected: ('127.0.0.1', 65104)
Received frame: SRC=AA:AA:AA:AA:01 DST=BB:BB:BB:BB:02 DATA=Hello
MAC Address Table:
AA:AA:AA:AA:01 -> Port 1
BB:BB:BB:BB:02 -> Port 2
>>>

Ln: 12 Col: 0
```

```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:\Users\thesa\Downloads\CO4_AT1_pycodes\Q4_client.py =====
Server: Frame received and forwarded
>>>

Ln: 6 Col: 0
```

```
Ln: 18 Col: 0
```

7. Result and Conclusion

The client successfully sends a simulated frame, the server receives it, updates/displays the MAC table, and returns a forwarding confirmation.

QUESTION 5

Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

1. Aim

To implement a StarSwitch class that manages active ports and simulates Ethernet broadcast behavior.

2. Algorithm

25. Create a StarSwitch object with an empty active-port list and broadcast log.
26. Use `add_port()` to connect hosts.
27. Use `show_ports()` to display currently active hosts.
28. Use `broadcast()` to send a frame to every active host except the source.
29. Store each broadcast operation in the log.
30. Use `remove_port()` to disconnect a host and verify the updated port list.

3. Python Program

```
class StarSwitch:
    def __init__(self):
        self.active_ports = []
        self.broadcast_log = []

    def add_port(self, host_id):
        if host_id not in self.active_ports:
            self.active_ports.append(host_id)
            print(f"{host_id} connected.")

    def remove_port(self, host_id):
        if host_id in self.active_ports:
            self.active_ports.remove(host_id)
            print(f"{host_id} disconnected.")

    def broadcast(self, frame, source_port):
        destinations = [
            host for host in self.active_ports
            if host != source_port
        ]

        entry = (
            f"Source: {source_port} | Frame: {frame} | "
            f"Destinations: {' '.join(destinations)}"
        )
        self.broadcast_log.append(entry)
        print(entry)

    def show_ports(self):
        print("Active ports:", self.active_ports)
```

```

switch = StarSwitch()

switch.add_port("Host1")
switch.add_port("Host2")
switch.add_port("Host3")
switch.add_port("Host4")

switch.show_ports()
switch.broadcast("DATA-101", "Host1")

switch.remove_port("Host3")
switch.show_ports()
switch.broadcast("DATA-102", "Host2")

```

4. Sample Output

```

Host1 connected.
Host2 connected.
Host3 connected.
Host4 connected.
Active ports: ['Host1', 'Host2', 'Host3', 'Host4']
Source: Host1 | Frame: DATA-101 | Destinations: Host2, Host3, Host4
Host3 disconnected.
Active ports: ['Host1', 'Host2', 'Host4']
Source: Host2 | Frame: DATA-102 | Destinations: Host1, Host4

```

Visual Representation

StarSwitch: Broadcast and Port Management

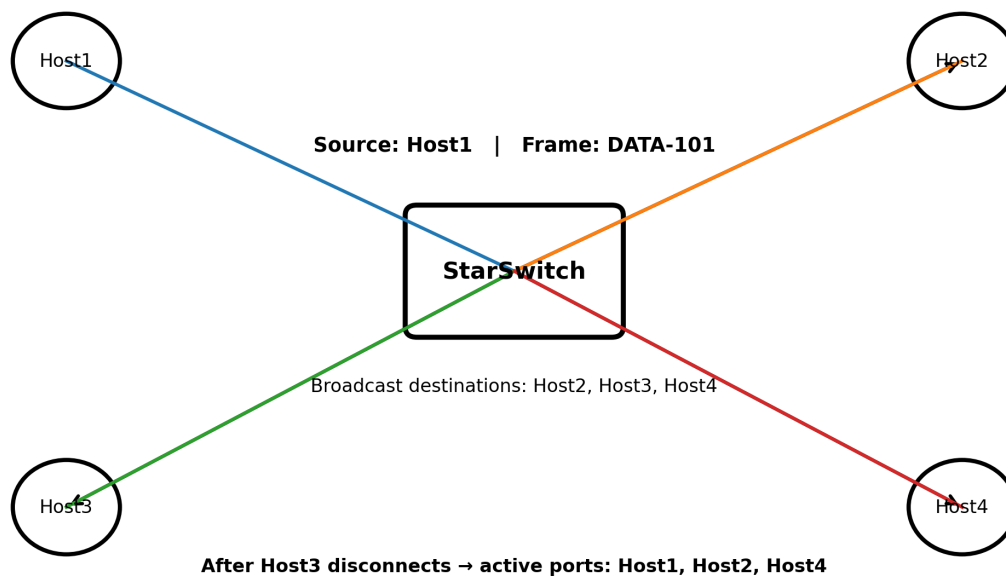
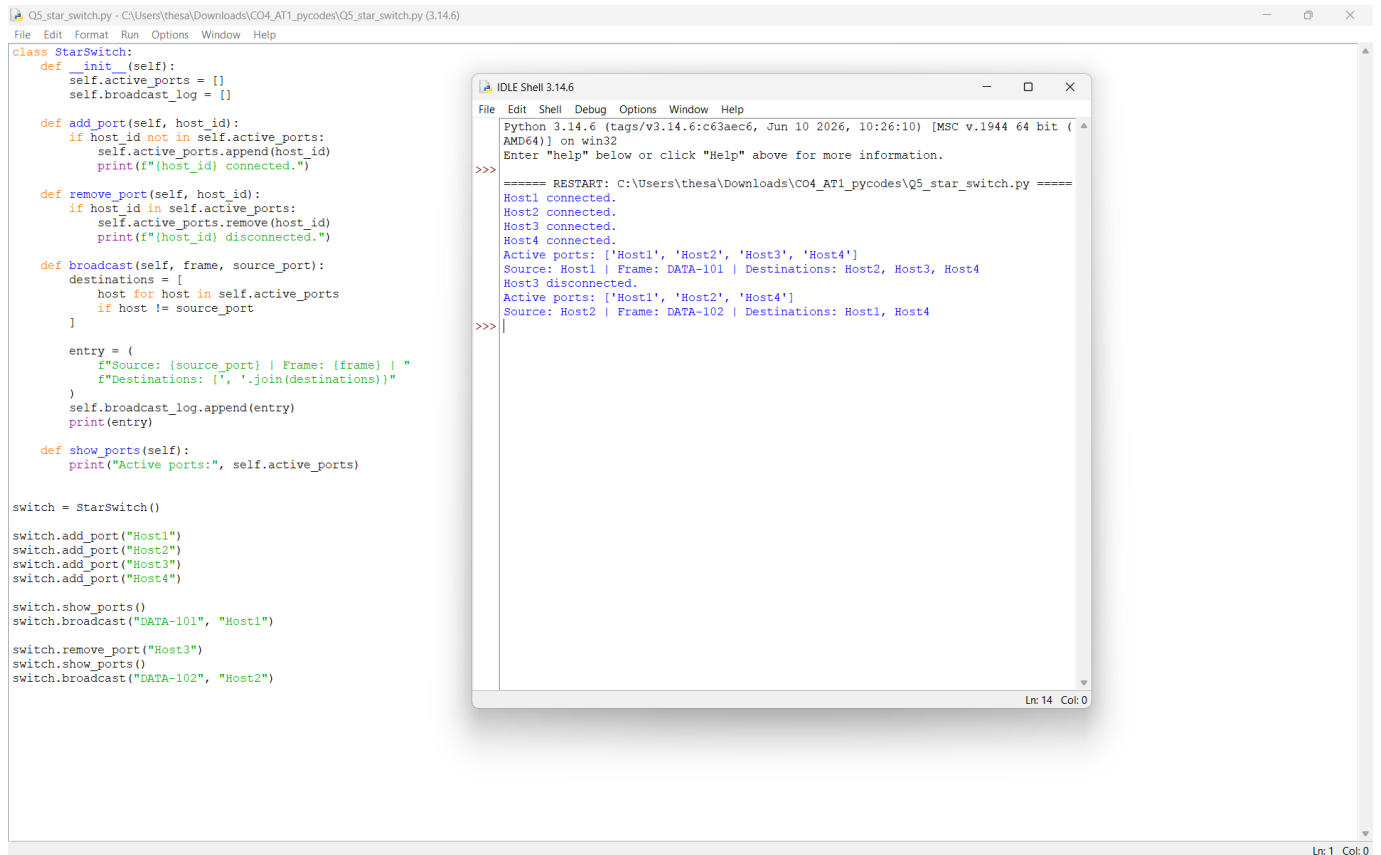


Figure 6. Visual representation of StarSwitch broadcast and port management.

5. Analysis

Broadcasting is useful when a sender does not yet know the destination port or when a frame is intended for all devices in the relevant broadcast domain. The simulation excludes the source port from the destination list and records the operation. The active-port list also demonstrates how a topology changes when a host disconnects.



The screenshot displays a Python IDE with two windows. The main window shows the `StarSwitch` class implementation, which manages a list of active ports and handles broadcast operations. The `__init__` method initializes `active_ports` and `broadcast_log`. The `add_port` method adds a host to the active ports, `remove_port` removes a host, and `broadcast` sends a frame to all active ports except the source. The `show_ports` method prints the current active ports. The execution at the bottom demonstrates adding four hosts, broadcasting to Host1, removing Host3, and broadcasting to Host2.

```
class StarSwitch:
    def __init__(self):
        self.active_ports = []
        self.broadcast_log = []

    def add_port(self, host_id):
        if host_id not in self.active_ports:
            self.active_ports.append(host_id)
            print(f"[{host_id}] connected.")

    def remove_port(self, host_id):
        if host_id in self.active_ports:
            self.active_ports.remove(host_id)
            print(f"[{host_id}] disconnected.")

    def broadcast(self, frame, source_port):
        destinations = []
        for host in self.active_ports:
            if host != source_port:
                destinations.append(host)

        entry = {
            "Source": source_port,
            "Frame": frame,
            "Destinations": destinations
        }
        self.broadcast_log.append(entry)
        print(entry)

    def show_ports(self):
        print("Active ports:", self.active_ports)

switch = StarSwitch()

switch.add_port("Host1")
switch.add_port("Host2")
switch.add_port("Host3")
switch.add_port("Host4")

switch.show_ports()
switch.broadcast("DATA-101", "Host1")

switch.remove_port("Host3")
switch.show_ports()
switch.broadcast("DATA-102", "Host2")
```

The output window shows the following execution results:

```
Python 3.14.6 (tags/v3.14.6:c63a6c6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:\Users\thesa\Downloads\CO4_AT1_pycodes\Q5_star_switch.py =====
Host1 connected.
Host2 connected.
Host3 connected.
Host4 connected.
Active ports: ['Host1', 'Host2', 'Host3', 'Host4']
Source: Host1 | Frame: DATA-101 | Destinations: Host2, Host3, Host4
Host3 disconnected.
Active ports: ['Host1', 'Host2', 'Host4']
Source: Host2 | Frame: DATA-102 | Destinations: Host1, Host4
>>>
```

6. Result and Conclusion

The `StarSwitch` class successfully adds and removes hosts, maintains active ports, broadcasts frames to all other connected hosts, and records each broadcast operation.